SLIIT
Discover Your Future

IT1170 – Data Structures and Algorithms

Lecture 11

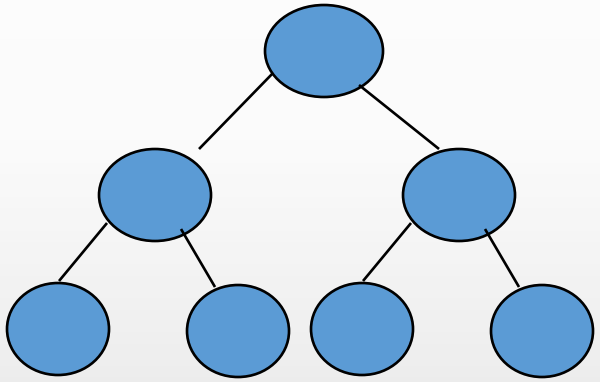
Heaps

Contents for Today

- **Tree**
- **Binary Tree**
- **Complete Binary Tree**
- **Heaps**
- **Heap Algorithms**
  - Maintaining Heap Property
  - Building Heaps
  - HeapSort Algorithms

Height of a Full Binary Tree

- A Full binary tree of height ***h*** that contains exactly $\mathbf{2^{h+1}-1}$ nodes

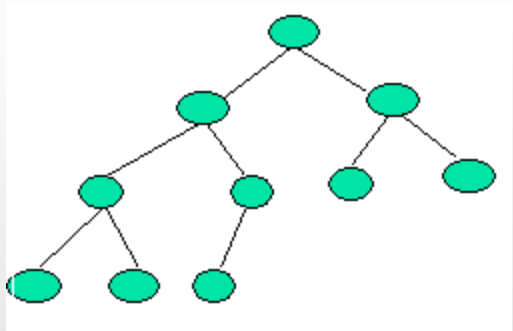


Height, ***h*** = 2, nodes = $2^{2+1}-1 = 7$

Height of a complete binary tree

Height of a complete binary tree that contains ***n*** elements is $\lfloor \log_2(n) \rfloor$

Example



- Above is a Complete Binary Tree with height = 3
- No of nodes: n = 10
- Height = $\lfloor \log_2(n) \rfloor = \lfloor \log_2(10) \rfloor = 3$

Heaps

Heap is an array object that can be viewed as a complete binary tree. There are two kinds of heaps

max heaps and **min heaps**

In both case, values in the nodes satisfy **Heap Property** which depend on the kind of heap

max-heap → max-heap property:

The value of each node is greater than or equal to those of its children.

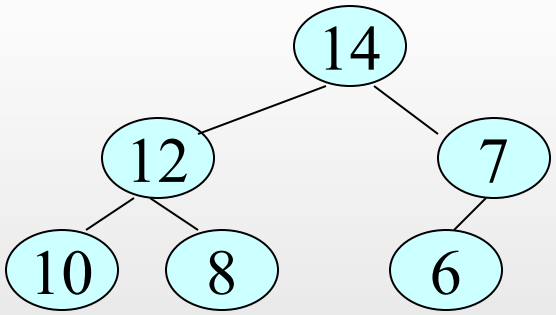
min-heap → min-heap property:

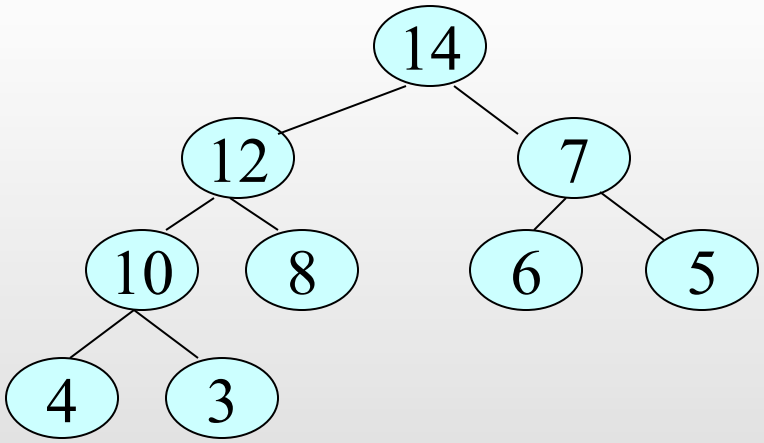
The value of each node is less than or equal to those of its children.

Max-heaps are used in heapsort algorithm

Heaps

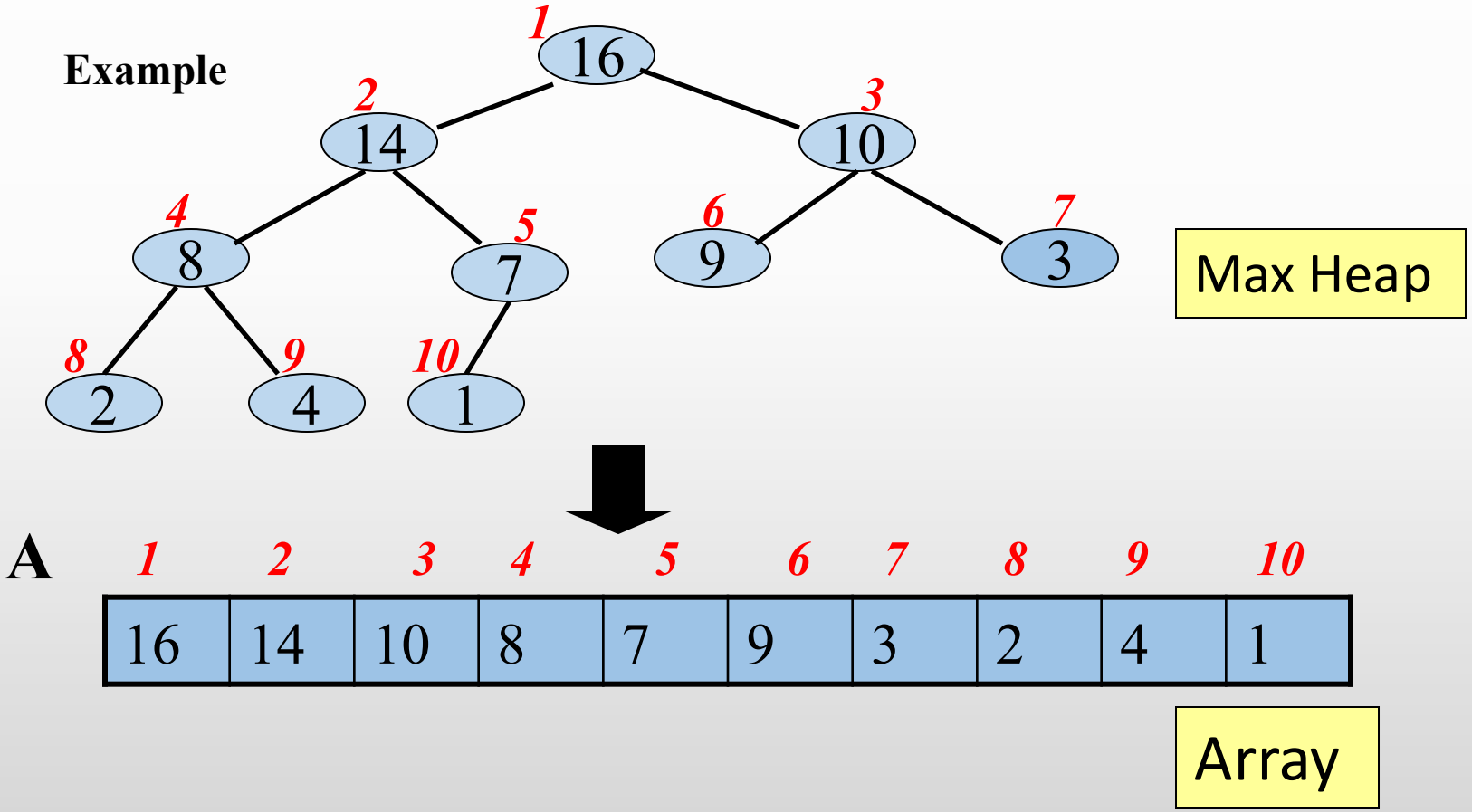
Complete Binary Tree with the **max-heap property - examples**





Heaps (contd.)

- A heap can be represented in a one-dimensional array



Heaps (contd.)

After representing a heap using an array **A**

- Root of the tree is **A[1]**

A	1	2	3	4	5	6	7	8	9	10
	16	14	10	8	7	9	3	2	4	1

Given node with index *i*,

- ***PARENT(i)*** is the index of parent of ***i***;

$$PARENT(i) = \lfloor i/2 \rfloor$$

- ***LEFT_CHILD(i)*** is the index of left child of ***i*** ;

$$LEFT_CHILD(i) = 2 \times i$$

- ***RIGHT_CHILD(i)*** is the index of right child of ***i***;

$$RIGHT_CHILD(i) = 2 \times i + 1$$

Heap Algorithms

- **MAX_HEAPIFY**:

  To maintain max-heap property

$$\boxed{A[PARENT(i)] \geq A[i]}$$

- **BUILD_MAX_HEAP**

  To build max-heap from an unsorted input array

- **HEAPSORT**

  Sorts an array in place.

MAX_HEAPIFY

- The MAX_HEAPIFY algorithm checks the heap elements for violation of the heap property and restores max-heap property;

$$\mathbf{A}[\mathit{PARENT}(i)] \geq \mathbf{A}[i]$$

- **Input**: An array A and index *i* to the array. *i* =1 if we want to heapify the whole tree. Subtrees rooted at *LEFT_CHILD(i)* and *RIGHT_CHILD(i)* are heaps

- **Output**: The elements of array A forming subtree rooted at *i* satisfy the heap property.

Maintaining the Heap Property

7	24	19	21	14	03	10	2	13	11

MAX_HEAPIFY (*A*,*i*)

```
l = LEFT(i);
r = RIGHT(i);
if l ≤ A.heap_size and A[ l ] > A[ i ]
        largest = l;
else largest = i;
if r ≤ A.heap_size and A[r] > A[largest]
        largest = r;
if largest ≠ i
        exchange A[i] with A[largest]
        MAX_HEAPIFY (A,largest)
```

Example

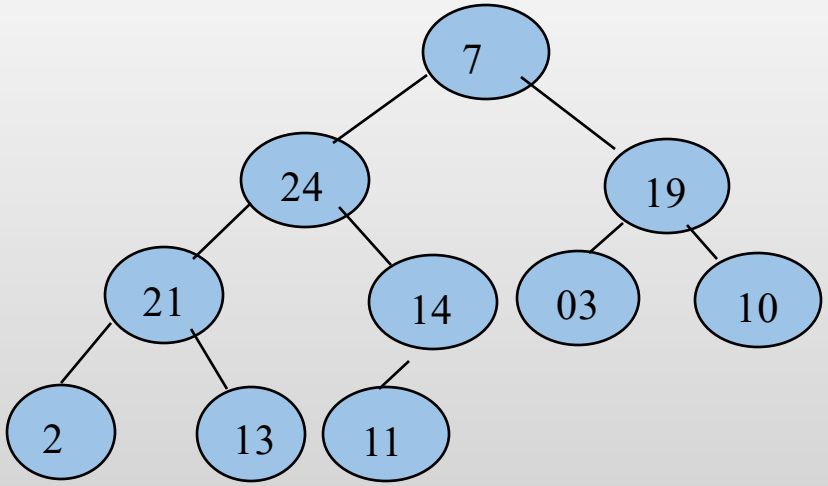
You are given the following array

A

1	2	3	4	5	6	7	8	9	10
7	24	19	21	14	03	10	2	13	11

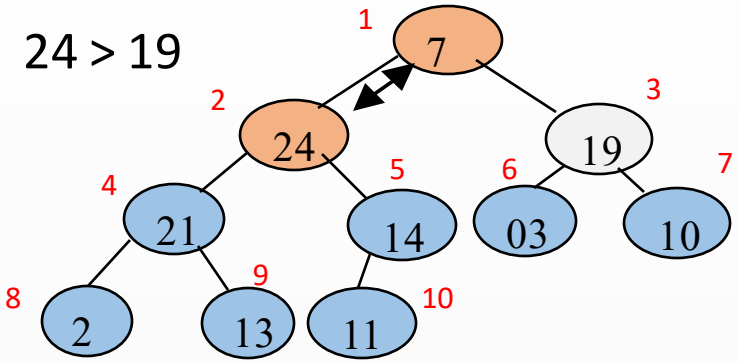
Now we are going to maintain the max-heap property

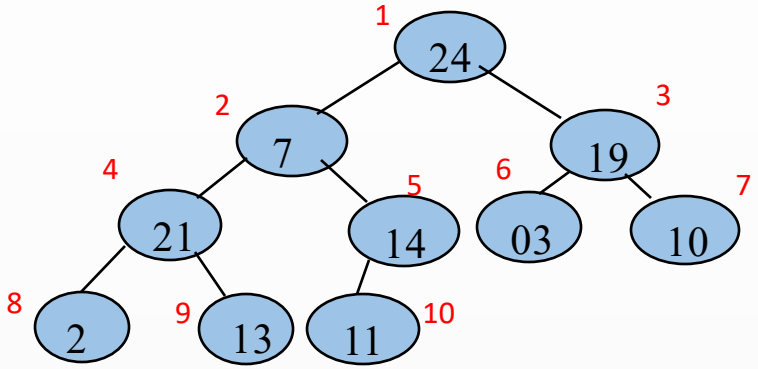
Drawing a heap would make our work easy



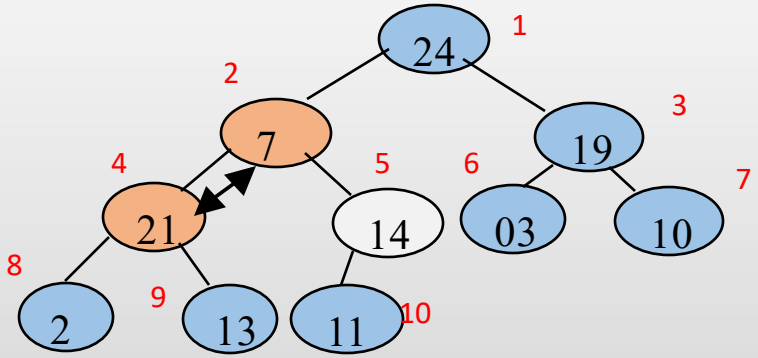
MAX_HEAPIFY (*A*,1)

24 > 19

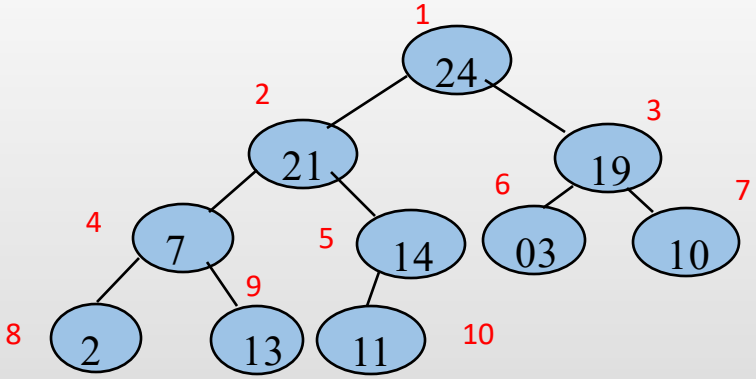




MAX_HEAPIFY (*A*,2)



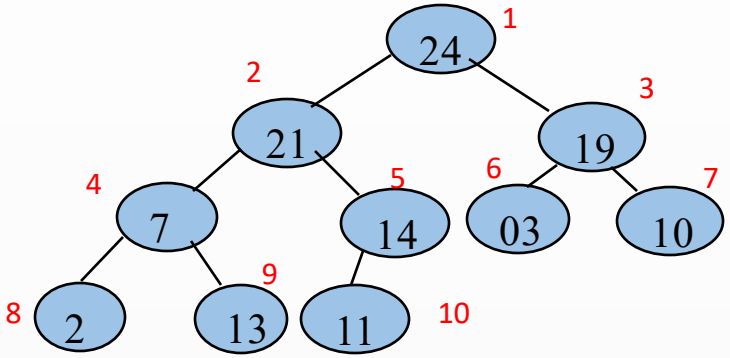
21 > 14

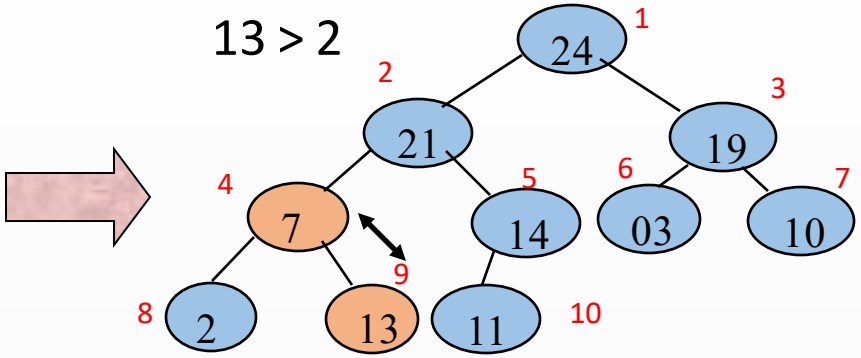


MAX_HEAPIFY (*A*,4)

To be contd.

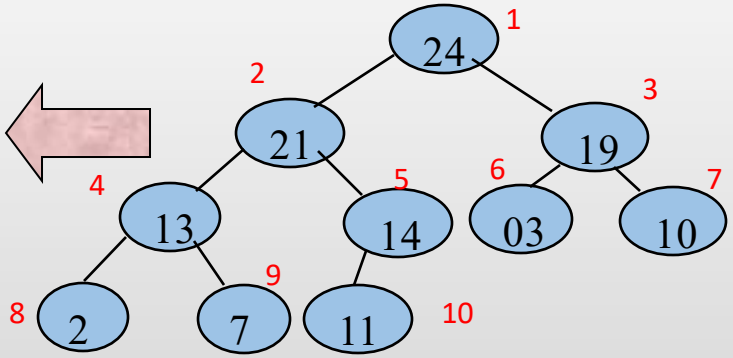
MAX_HEAPIFY (*A*,1) (contd.)





MAX_HEAPIFY (*A*,4)

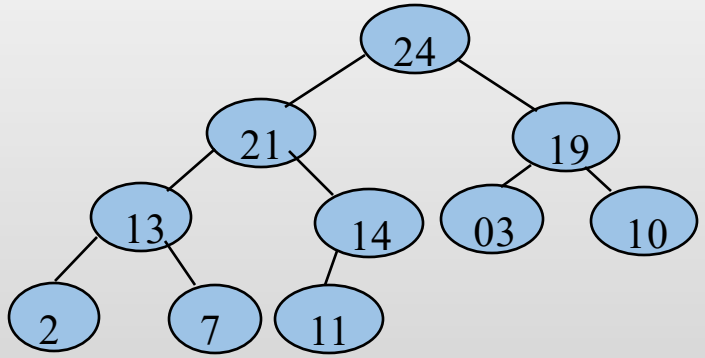
<u>Important point</u> Although we represent this process using a heap actually all the task in done on the input array



Resulting Heap

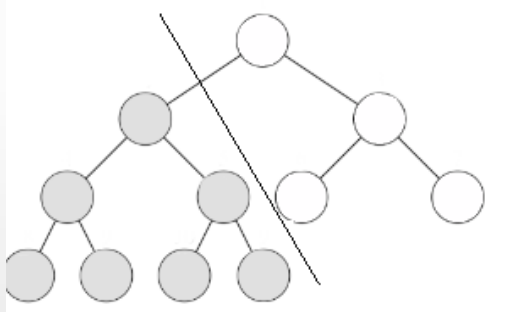
Array view of MAX_HEAPIFY Algorithm

7	24	19	21	14	03	10	02	13	11
24	7	19	21	14	03	10	02	13	11
24	21	19	07	14	03	10	02	13	11
24	21	19	13	14	03	10	02	07	11



Analysis of Heapify Algorithm.

- The running time of MAX-HEAPIFY on a subtree of size *n* rooted at given node *i* is the Θ(1) time plus the time to run MAX-HEAPIFY on a subtree rooted at one of the children of node *i*.
- The children's subtrees -the worst case occurs when the last row of the tree is exactly half full



- Alternatively, we can characterize the running time of MAX-HEAPIFY on a node of height *h* as $O(h)$.

The solution to this recurrence is

$$T(n) = O(\lg n)$$